



HACKFEST 2026

Problem Statement Documentation & Technical Submission

Team ID	KH070
Team Name	QuadraForge
Team Leader Name	Ankita Pawar
Problem Statement No.	PS20
Problem Statement Name	Agentic Disaster Relief & Emergency Resource Coordinator
Date of Submission	11 September 2026

SECTION 1 — TEAM DETAILS

Team ID

KH070

Team Name

QuadraForge

Team Leader Name

Ankita Pawar

Team Members Name(s)

1. Tanvi Biradar
2. Janhavi Gawandgave
3. Vaishnavi Kinhale

Problem Statement Number

PS20

Problem Statement Name

Agentic Disaster Relief & Emergency Resource Coordinator

SECTION 2 — UNDERSTANDING OF THE PROBLEM

Problem Statement

During a disaster or emergency, response teams must make decisions using rapidly changing information about affected zones, resource requirements, resource availability and ongoing response activities. Multiple agencies may operate simultaneously, making it difficult to coordinate resources effectively and avoid overlapping efforts.

The key challenge is to intelligently assess incoming disaster information, identify and prioritize affected zones, match available resources with emergency requirements, generate alerts for critical situations, and continuously update resource decisions when new reports or changing conditions are received.

Therefore, the problem is to develop an **Agentic Disaster Relief & Emergency Resource Coordinator** that uses AI-based decision support, backend resource management and an interactive command-center interface to improve disaster response coordination, resource utilization and timely decision-making.

Agentic AI, resource allocation, dynamic reassessment, resource reallocation, multi-agency coordination, duplicate-effort detection, and what-if simulation

Problem Analysis / Background

Disaster relief operations involve multiple affected zones, limited resources and different response agencies working under rapidly changing conditions. Information such as disaster type, affected population, severity, required resources and available resources can change as new reports are received.

A static resource-allocation approach may become ineffective when the situation changes. For example, a zone that was initially assigned a lower priority may become critical after a new emergency report, requiring the existing resource allocation to be reconsidered. Similarly, multiple agencies may independently respond to the same requirement, resulting in potential duplicate efforts while another affected zone may have an unmet requirement.

The system therefore needs to maintain information about disasters, affected zones, resource inventory and availability, previous allocations and new reports. It must also provide decision-support information such as priority levels, resource recommendations, alerts, reallocations and explanations for decisions.

The major stakeholders include disaster-management authorities, emergency-response teams, rescue and medical teams, relief agencies and other organizations involved in resource distribution, as well as the affected population that depends on timely relief.

Proposed Solution

We propose an Agentic Disaster Relief & Emergency Resource Coordinator, an intelligent disaster-response coordination platform that combines an AI agent, backend resource management, MongoDB-based data storage and an interactive command-center interface.

The system will receive structured information containing disaster details, affected zones, available resources, previous allocations and new reports. The AI agent will analyze this information and return structured results including prioritized zones, resource allocations, alerts, reallocation requirements, recommended reallocations, duplicate efforts and explanations.

Each affected zone will receive a priority classification of CRITICAL, HIGH, MEDIUM or LOW, along with a numerical priority score from **0 to 100**. Resource allocation recommendations will identify the resource, target zone, resource type, quantity, unit, urgency and reason for the allocation.

The backend will manage the disaster information, affected-zone data, resource inventory and allocation information using MongoDB, while the frontend will provide a centralized disaster command-center interface. The interface will include Dashboard/Overview, Disaster Input, Affected Zones, Resource Status, AI Relief Plan, Critical Alerts, Resource Allocation, What-if Simulation and Allocation History, along with loading and error states.

A key feature of the proposed system is dynamic reassessment. When a new report or hypothetical situation changes the current state, the system can evaluate the changed priorities and determine whether resource reallocation is required. The What-if Simulation will allow a hypothetical scenario to be evaluated by comparing the current and resulting situation, including priority changes, reallocation requirements, recommended resource changes and impact.

The system will also identify potential duplicate efforts among response activities and maintain allocation history to improve coordination and traceability. Through this combination of AI-based assessment, resource management, simulation and centralized visualization, the proposed solution aims to support faster, more coordinated and adaptive disaster-relief decisions.

SECTION 3 — RESEARCH

Research Conducted

Our team researched the major challenges in disaster-relief coordination, particularly rapidly changing emergency conditions, prioritization of affected zones, limited relief resources, resource allocation, coordination between multiple agencies and the need for continuous reassessment.

We studied the use of AI-based and multi-agent approaches for emergency decision support, where different tasks can be handled by specialized components. We also examined approaches for converting disaster reports into structured information, assigning priority scores, matching resource requirements with available resources, generating alerts and supporting dynamic reallocation.

Based on this research, our solution uses a structured AI-agent contract in which the AI receives disaster information, affected zones, available resources, previous allocations and new reports, and produces prioritized zones, allocations, alerts, reallocation recommendations, duplicate-effort detection and explanations.

We also researched interactive disaster command-center interfaces and incorporated features such as affected-zone monitoring, resource status, AI recommendations, critical alerts, resource allocation, what-if simulation and allocation history.

Existing Solutions / Similar Work

Existing disaster-management and emergency-response systems provide capabilities such as incident reporting, emergency information, mapping, monitoring and coordination. These systems demonstrate the importance of centralized information and timely communication during disaster response.

However, our proposed solution focuses specifically on the intelligent coordination and decision-support layer. Instead of only displaying disaster information, our system is designed to analyze the current state, prioritize affected zones, recommend resource allocations, identify potential duplicate efforts and reassess decisions when new information is received.

The proposed solution further introduces what-if simulation, allowing response teams to evaluate a hypothetical change in the disaster situation and observe its effect on priorities and resource requirements. It also provides explanations for AI recommendations and maintains allocation history for traceability.

Therefore, the key improvement of our solution is the combination of agentic AI, dynamic reassessment, resource reallocation, duplicate-effort detection, explainable recommendations and what-if simulation within a unified disaster-relief coordination workflow.

References / Sources

1. United Nations Office for Disaster Risk Reduction (UNDRR), “Sendai Framework for Disaster Risk Reduction 2015–2030.”
<https://www.undrr.org/publication/sendai-framework-disaster-risk-reduction-2015-2030>
2. United Nations Office for Disaster Risk Reduction (UNDRR), “What is the Sendai Framework for Disaster Risk Reduction?”
<https://www.undrr.org/implementing-sendai-framework/what-sendai-framework>
3. NASA Disasters Program, “Applying Satellite Data to Support Disaster Response and Emergency Management Decision Making.”
<https://ntrs.nasa.gov/citations/20240013653>

- NASA Disasters Program, “Practitioner Resources.”
<https://disasters.nasa.gov/what-we-do/disasters/practitioner-resources>

SECTION 4 — TECHNICAL DOCUMENTATION

Technology Stack

Frontend: HTML, CSS, JavaScript

Backend: Node.js, Express.js

Database: MongoDB

AI/Agent Layer: AI-based agent with structured JSON input/output

AI Integration: Gemini/Groq API, depending on the final AI implementation

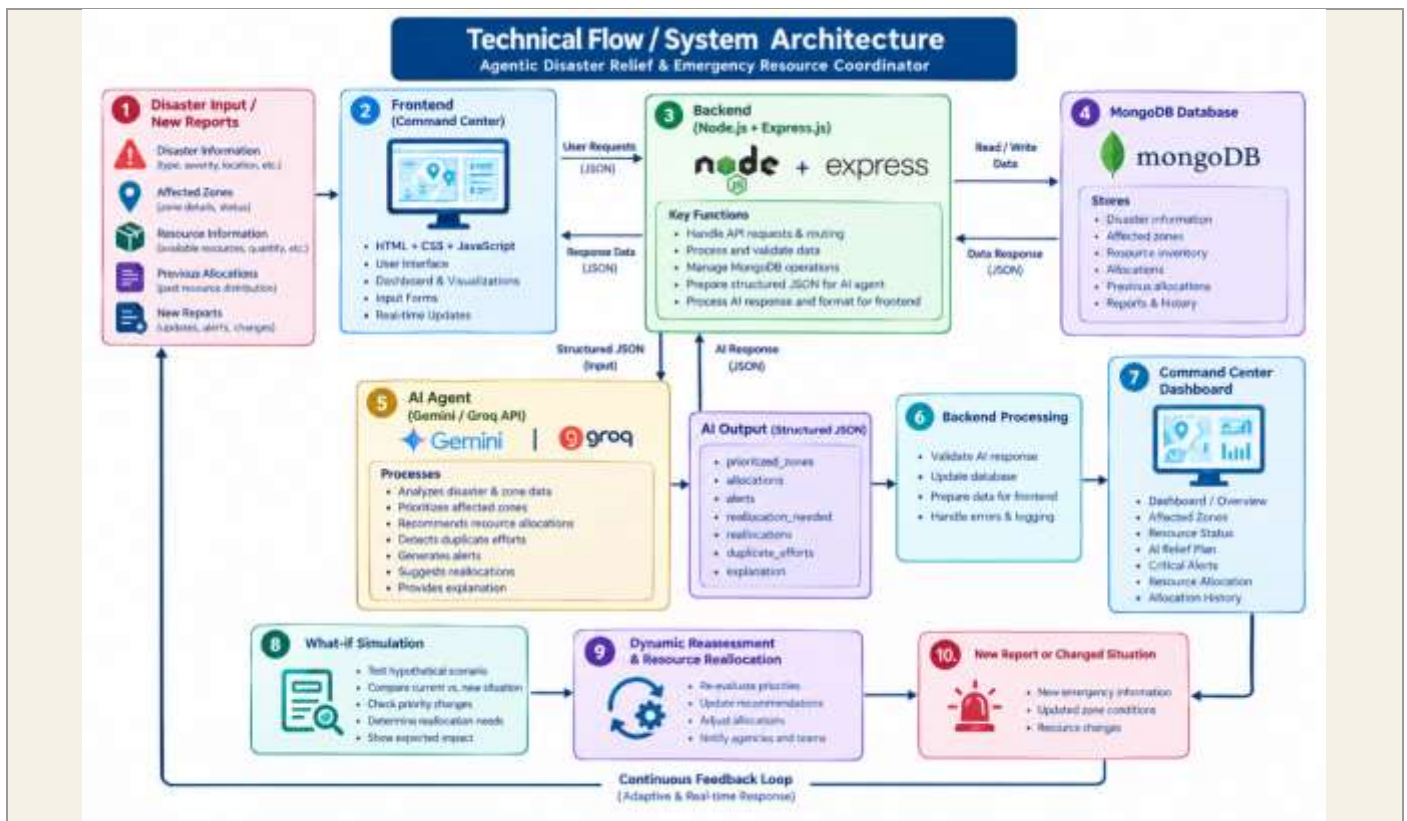
Data Exchange Format: JSON

Version Control: Git and GitHub

Development Environment: Visual Studio Code

The system follows a modular architecture in which the frontend provides the disaster command-center interface, the Node.js and Express.js backend manages application logic and communication with MongoDB and the AI agent, and the AI agent provides structured recommendations for prioritization, resource allocation, alerts and reassessment.

Technical Flow / System Architecture Diagram



Working / Implementation Approach

The system follows a modular workflow connecting the frontend, backend, database and AI agent.

1. **Disaster Data Input:**
The user enters disaster information and emergency reports through the command-center frontend. The input may contain disaster details, affected zones, zone conditions, resource requirements and new situation updates.
2. **Backend Processing:**
The frontend communicates with the Node.js and Express.js backend. The backend processes the incoming data and manages communication between the frontend, MongoDB and the AI agent.
3. **Data Management:**
MongoDB stores and provides the required disaster information, affected zones, resource inventory, previous allocations, reports, alerts and allocation history.
4. **AI Agent Input:**
The backend prepares a structured JSON object containing:
 - disaster
 - zones
 - resources
 - previous_allocations
 - new_reports

This structured information is provided to the AI agent for situation analysis.

5. **AI-Based Analysis:**
The AI agent analyzes the current disaster situation and generates structured recommendations. It identifies zone priorities, recommends resource allocations, generates alerts, detects potential duplicate efforts and determines whether resource reallocation may be required.
6. **AI Response:**
The AI returns structured JSON containing:
 - prioritized_zones
 - allocations
 - alerts
 - reallocation_needed
 - reallocations
 - duplicate_efforts
 - explanation
7. **Backend Integration:**
The backend processes the AI response and prepares the results for the frontend. The system maintains compatibility between the AI contract, MongoDB data and frontend requirements.
8. **Command-Center Dashboard:**
The frontend displays the processed results through the disaster command center, including affected zones, resource status, AI relief recommendations, critical alerts, resource allocation and allocation history.
9. **What-if Simulation:**
The system supports hypothetical scenarios by comparing the current state with a proposed situation. The AI evaluates changes in priority, determines whether reallocation is required, recommends resource changes and provides the expected impact.
10. **Dynamic Reassessment:**
When a new emergency report, changed zone condition or resource change is received, the updated information is processed again. The system reassesses priorities and resource recommendations so that response decisions can adapt to the changing situation.
11. **History and Traceability:**
Allocation history is maintained so that previous resource-distribution decisions and changes can be reviewed. This provides greater visibility and traceability of the response process.

Overall, the implementation follows the flow:

Disaster Input → Frontend → Node.js/Express.js Backend → MongoDB + AI Agent → AI Recommendations → Backend Processing → Command-Center Dashboard → What-if Simulation / Dynamic Reassessment → Updated Response

AI/ML Details (if applicable)

The proposed system uses an AI-agent-based approach for disaster situation analysis and emergency resource decision support.

The AI agent receives a structured JSON object containing the current disaster information, affected zones, available resources, previous allocations and new emergency reports.

AI Input:

- disaster
- zones
- resources
- previous_allocations
- new_reports

The AI agent analyzes the provided information and returns a structured JSON response containing:

AI Output:

- prioritized_zones
- allocations
- alerts
- reallocation_needed
- reallocations
- duplicate_efforts
- explanation

Affected zones are assigned a priority level of CRITICAL, HIGH, MEDIUM or LOW, along with a priority score ranging from 0 to 100.

Resource allocation recommendations include:

- resource_id
- zone_id
- resource_type
- quantity
- unit
- reason
- urgency

Alerts include:

- alert_id
- type
- severity
- zone_id
- message

The system also supports what-if simulation, in which the current state and a hypothetical scenario are provided to the AI agent. The resulting analysis identifies changes in priority, whether reallocation is required, recommended resource changes and the expected impact.

The AI response is returned in structured JSON format so that the Node.js and Express.js backend, MongoDB database and frontend can maintain a consistent data structure.

The AI component is used for situation analysis, prioritization and decision-support recommendations. The project does not claim custom model training unless a trained machine-learning model is incorporated into the final implementation.

GitHub Repository Link

<https://github.com/janhavi-codes18/Ps20-disaster-relief>

SECTION 5 — ORIGINALITY & PLAGIARISM DECLARATION

- ✓ I confirm that the solution submitted by our team is developed during the hackathon. We have not submitted a pre-built project as our primary solution. Any open-source code, APIs, datasets, libraries, templates, or AI-assisted development used by us have been appropriately acknowledged.

Open-Source / External Resources Used

Node.js, Express.js, MongoDB, HTML, CSS, JavaScript, Git, GitHub, AI API documentation and publicly available disaster-management references.

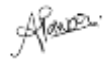
AI Tools Used

AI tools were used as development assistance for technical guidance, debugging, documentation and implementation support. The team is responsible for the final design, implementation, testing and integration.

SECTION 6 — UPLOADS / ATTACHMENTS CHECKLIST

- ✓ Technical Flow / Architecture Diagram
- ✓ Research / Reference Document (if applicable)
- ✓ Project Documentation / Technical Report
- ✓ GitHub Repository Link
- ☐ Screenshots of Working Prototype
- ☐ Dataset / Sample Data (if applicable)
- ☐ Additional Diagrams (if any)

Team Leader Signature:

A handwritten signature in black ink, appearing to read "Aman".

Date: 11-09-2026